

Study Buddy: A Dual-Agent AI Education System

Project Proposal for UCS503P

Submitted to: Ms. Nisha Thakur

Thapar Institute of Engineering and Technology

August 15, 2026

1 Higher-order goal

... secondary framing

The main idea behind this project is to build something that actually supports the Feynman Technique — learning by teaching — instead of yet another Q&A chatbot. Instead of the AI playing tutor, we flip the roles: the user teaches, and a separate “Child Agent” has no access to the source material and is evaluated using only the structured knowledge extracted from the teaching session. A second “Teacher Agent” holds the real source material and is used to check how much the Child Agent actually picked up. The engineering goal, put plainly, is to see whether “did the student really understand this topic” can be turned into something we can measure automatically, rather than something we have to take on faith.

2 Time-to-value

... primary heuristic

- We are prototyping with pretrained LLMs (Llama 3.1 / GPT-class models) rather than training anything ourselves, so we can get an actual teacher–child interaction loop running quickly and see if the core idea holds up before investing in anything more elaborate.
- The memory/learning module is being built early — even in a rough form — because the whole point of the project lives there. We would rather find out in week one whether the Child Agent retains anything useful than discover it at the end.
- We are treating “can the Child Agent pass a closed-book exam using only what the student taught it” as the concrete success signal, mainly because it is something we can score automatically instead of arguing about.

3 Problem Statement

Most educational AI tools put the model in the role of an all-knowing tutor, which tends to encourage passive reading rather than active thinking. And when students are asked to explain a concept back — which is usually the best way to check understanding — typical chatbots cannot convincingly act like a learner who knows nothing, because they already know everything. A few specific pain points motivated this project:

- **Passive engagement:** Students mostly read or listen, rather than having to construct an explanation themselves.
- **No real knowledge isolation:** A standard chatbot cannot “forget” its pretraining, so it is hard to tell whether it understood the student or just already knew the answer.
- **Evaluation is subjective:** Without some structure around it, judging whether a student “explained something well” is mostly a gut call.

Why this matters now: As LLMs make raw information easier than ever to retrieve, the harder and more useful problem is checking whether someone actually understood what they read, not just that they read it.

4 Proposed Solution

... Software Proposition

4.1 Overview

We propose building “Study Buddy,” a two-agent system where each agent has a clearly different role and a different set of permissions, tied together by a shared memory/learning layer.

- **Teacher Agent (expert knowledge):** Has access to the uploaded study material and is responsible for generating test questions and grading the Child Agent’s answers against that material.
- **Child Agent (learns from the student):** Deliberately kept away from the source material. It uses a smaller/weaker LLM than the Teacher Agent where practical and updates its structured knowledge only through the teaching conversation with the student.
- **Model Separation:** The Teacher Agent will use a stronger model for question generation and grading, while the Child Agent will use a smaller/weaker model to reduce the amount of pretrained knowledge available for unintended leakage.
- **Memory & learning engine:** The part of the project we are most unsure of and most interested in. It pulls structured knowledge (concepts, definitions, how confident it is) out of the teaching session, and that structured record becomes the Child Agent’s entire knowledge base for the exam.

4.2 Core workflow

1. **Knowledge preparation:** The user uploads study material (PDFs/notes); the Teacher Agent processes this as the ground truth.
2. **Cold baseline evaluation:** Test the Child Agent before teaching to measure its pre-existing knowledge.
3. **Teaching session:** The user explains the material to the Child Agent, by text (and later, voice).
4. **Memory construction:** The learning engine extracts structured facts from that session and updates the Child Agent’s memory.
5. **Test generation and execution:** The Teacher Agent writes an exam from the source material, and the Child Agent takes it closed-book, using only its own memory.
6. **Evaluation:** The Teacher Agent grades the Child Agent’s answers against the ground truth, and the result is shown back to the student as a measure of how well they taught the topic.

5 Solution Approach

... Engineering focus

We use pretrained models as the reasoning engine rather than training or fine-tuning models from scratch, allowing us to focus on the system architecture, role isolation, memory persistence, and multi-agent orchestration that form the core of our approach.

5.1 System architecture

- **Frontend:** React.js / Next.js, for the upload portal, chat interface, and results dashboard.
- **Backend & AI integration:** Python with FastAPI, handling agent prompting, function calling, and keeping the two agents' contexts properly separated.
- **Memory & data layer:** PostgreSQL for application data, and FAISS/ChromaDB for the Child Agent's own retrieval index.
- **Document & voice processing:** PyPDF for ingesting study material, and the Whisper API for speech-to-text once we get to the voice-based teaching mode.
- **Model allocation:** The Teacher Agent uses a stronger LLM, while the Child Agent uses a smaller/weaker LLM to reduce knowledge leakage.

None of this is meant to be a novel stack — it is a set of fairly standard, well-supported tools chosen so we can spend our time on the agent design instead of infrastructure.

5.2 The learning engine

...non-research innovation

This is the one genuinely uncertain part of the project, and honestly the part we are most curious about. Rather than updating any model weights, “learning” here just means turning the raw teaching conversation into structured data:

- For each evaluation document, a canonical list of key concepts, definitions, and relationships will be prepared from the source material. This list will serve as the reference for measuring memory extraction accuracy.
- A concept-extraction step turns the transcript into structured JSON — concept, dependencies, and a rough confidence score.
- For the initial evaluation set, the reference concepts will be manually reviewed and annotated. If LLM-assisted generation is used to accelerate annotation, the generated reference will be manually verified and its potential annotation bias will be acknowledged as a limitation.
- The original conversation is then discarded, so the Child Agent is forced to answer exam questions using only that structured memory, not the raw chat log.

We do not yet know how well this extraction step will hold up on messier, less organized explanations — that is one of the things we expect to learn from building the first version.

6 Evaluation Criterion

...measurable and attributable

6.1 Primary evaluation metric

Teaching Efficacy Score (TES): The score that the Child Agent gets from the Teacher Agent on the generated exam.

- **Target:** Show a clear, positive relationship between how completely a student taught a topic and the Child Agent's exam score — i.e., better teaching should visibly translate into a better score, not just a random one.
- **Attribution:** Measured directly from the Teacher Agent's automated grading output, logged per session so we can compare teaching quality against score over multiple sessions.

6.2 Secondary evaluation metrics

- **Memory extraction accuracy:** Percentage of concepts from the canonical reference concept list that are correctly captured in the Child Agent’s structured memory.
- **Cold baseline score:** The Child Agent’s score on an exam administered before any teaching-derived memory is provided. This provides a measurable estimate of knowledge the Child Agent can demonstrate without the student’s teaching.
- **Knowledge leakage indicator:** The cold baseline score will be used as an indicator of potential pretrained knowledge leakage. A lower baseline relative to the post-teaching score provides stronger evidence that the student’s teaching contributed to the Child Agent’s performance.
- **System latency:** Latency will be measured separately for transcription, memory extraction, retrieval, and LLM generation. The goal is to maintain an interactive response time. An end-to-end response time below 2 seconds will be treated as an aspirational target rather than a strict requirement because the system involves multiple chained model/API calls.

7 Scalability

... with foresight

1. **Scalable (theoretically):** Because the Child Agent’s “brain” is really just a database query against its memory store, memory can grow without running into LLM context-window limits — the reasoning step stays stateless regardless of how much has been taught.
2. **Operationally deployable (practically):** Dockerized backend services on AWS or Render, with managed Postgres and vector stores that can scale as usage grows..

8 Engine availability heuristic

A mature set of tools and services is available to implement Study Buddy without requiring the development of new infrastructure:

- **Pretrained LLMs:** Llama 3.1 / GPT-class models can provide the reasoning capabilities required by the Teacher and Child Agents.
- **Backend framework:** Python with FastAPI can handle API requests, agent orchestration, prompting, and communication between system components.
- **Database and memory:** PostgreSQL can manage application data, while FAISS or ChromaDB can provide vector-based retrieval for the Child Agent’s memory.
- **Document processing:** PyPDF provides a practical way to extract and process content from uploaded study material.
- **Voice processing:** The Whisper API can be integrated for speech-to-text when the voice-based teaching mode is introduced.
- **Frontend and analytics:** React.js / Next.js can provide the teaching interface and results dashboard, with Chart.js supporting visualisation of learning and evaluation metrics.

These are established and well-supported technologies, allowing the project to focus primarily on the multi-agent design, knowledge isolation, memory construction, and evaluation rather than building infrastructure from scratch.

9 Project scope and deliverables

9.1 Initial deliverable

...first iteration

- Document upload and Teacher Agent ingestion pipeline.
- Text-based teaching interface for the Child Agent.
- A first-pass memory extraction algorithm (transcript $\rightarrow$ structured JSON).
- Basic exam generation and automated grading workflow.

9.2 Subsequent deliverables

- Voice integration via Whisper for a more natural, spoken teaching session.
- A proper analytics dashboard (Chart.js) so students can see how their TES changes over time.
- Confidence-based follow-up questions, where the Child Agent asks for clarification when its confidence in a concept is low, instead of silently guessing.

10 Risks and mitigations

- **Pretrained knowledge leakage:** The Child Agent may rely on pretrained knowledge instead of the student's teaching. Mitigation: use a weaker model, isolate it from source material, restrict retrieval to its memory, and use a cold baseline to measure leakage.
- **Context window limits:** Long teaching sessions could blow past token limits. Mitigation: the memory engine digests and compresses the conversation into the database as it goes, rather than holding on to the full raw chat log.
- **Uncertain extraction quality:** The concept-extraction step is new and untested on real conversational teaching data, so early accuracy may be inconsistent. Mitigation: Start with a small, well-defined evaluation set, use canonical concept lists as ground truth, measure extraction accuracy, and iteratively improve the extraction process.

11 Summary

Study Buddy tries to flip the usual AI-tutor setup: instead of the model teaching the student, the student teaches the model, and we grade what actually got through. By pairing a Teacher Agent and a Child Agent around a shared memory layer, the project gives us a concrete, measurable way to test the Feynman Technique rather than just asserting it works. We are not trying to advance language modeling itself — we are using existing LLM and RAG techniques to build a specific, testable learning tool, and treating the memory/extraction layer as the part worth getting right.